

CUMCM LaTeX 实战入门

零基础 · 边做边学 · 直接拿 cumcm 模板写一篇数学建模论文

这本手册把「cumcm 模板」当成教材。你不需要任何 LaTeX 基础——只要有一台电脑，跟着每一章动手做：打开对应文件、粘贴代码、编译、看效果。做完这本，你就掌握了用 LaTeX 写一篇数学建模竞赛论文所需的全部常用能力。每一步都按「操作 → 代码 → 说明」给全，照着做就行。

第零章 开工前先弄明白三件事

别急着写代码。先用 5 分钟搞懂三件事，后面就顺了。

0.1 LaTeX 是什么？它和 Word 有什么不同？

你平时用 Word，是「所见即所得」：你看到的排版，就是最终的样子。LaTeX 不同，它分两步：

- 第一步：写「源文件」（后缀 `.tex` 的纯文本），里面是你想写的内容 + 排版命令。
- 第二步：用编译器把 `.tex` 编译成一个 `.pdf` 成品。

打个比方：`.tex` 就像剧本，编译就是把这个剧本拍成电影（`.pdf`）。你只写剧本，排版是模板和 LaTeX 自动替你做的。

这就是为什么数学建模论文要用 LaTeX：公式漂亮、图和表自动编号、交叉引用自动更新、整个比赛统一格式——这些正是 Word 的痛点。cumcm 模板已经把这些格式都做好了，你只需要往里填内容。

0.2 这个模板里每个文件是干什么的？

先看一眼 cumcm 项目长什么样（对着你的文件夹看）：

cumcm/	
├ main.tex	主文档（编译入口，最重要，从这里开始）
├ latexmkrc	latexmk 配置（让编译更省心，别动）
├ README.md	模板说明（必读）
├ commons/	模板本体（建议不要改）
└ cumcmthesis.cls	文档类：负责所有格式
└ preamble.tex	你的导言区：加载宏包、自定义命令（可加自己的东西）

└─ contents/	内容区（你写论文的地方）
│ └─ info.tex	题目 + 队员信息（封面用）
│ └─ abstract.tex	摘要 + 关键词
│ └─ references.tex	参考文献
│ └─ sections/	正文（s1.tex ... s6.tex）
│ └─ appendix/	附录（a1.tex ...）
└─ figures/	放图片（pdf / jpg / png）
└─ fonts/	模板自带的字体（times、arial）
└─ docs/	竞赛通知与格式规范 PDF

文件/目录	你的角色	要不要动？
main.tex	总指挥：把各部分串起来	要动（添加/删减章节）
commons/cumcmthesis.cls	负责所有全局格式	别动
commons/preamble.tex	加载额外宏包、自定义命令	可加东西
contents/info.tex	题目、题号、学校、队员、指导教师、日期	要填
contents/abstract.tex	摘要 + 关键词	要写
contents/sections/*.tex	论文正文，一个文件一个章节	要写
contents/references.tex	参考文献列表	要填
contents/appendix/*.tex	附录（代码、算法等）	要写
figures/	所有插图图片	要放

记住一句话：main.tex 负责「拼装」，contents/ 负责「内容」，commons/ 负责「格式」。你主要写 contents/，偶尔改 main.tex。

0.3 装环境（一次性，装好就不用再碰）

► 安装 TeX Live（必须，提供了编译器 xelatex）

- 到 TeX Live 官网下载安装包，Windows / Linux 均可；macOS 装 MacTeX。
- 安装时选「完整安装」最省心（如果磁盘紧张，至少选 scheme-small + 常用宏包）。
- 装完后打开终端（Windows 可开 PowerShell / cmd），输入 `xelatex --version`，回车；能显示版本号就成功了。

► 安装 VS Code 和 LaTeX Workshop 插件（写代码 + 一键编译）

- 到 VS Code 官网下载安装。
- 打开 VS Code → 左侧扩展图标（Ctrl+Shift+X）→ 搜索 LaTeX Workshop（作者 James Yu）→ 安装。

► 打开模板文件夹

- VS Code 菜单「文件 → 打开文件夹」，选中 `cumcm` 这个文件夹。

`cumcm` 模板在文档类里强制要求用 `xelatex`。如果你用 `pdflatex` 编译会直接报错，这是正常的「防呆保护」。以后统一用 `xelatex` 或 `latexmk`。

第一章 第一步：把模板编译出来

目标：把项目自带的 `main.tex` 编译成 PDF，你第一次看到「论文长什么样」。做完这一章，你就走通了 LaTeX 的完整流程。

1.1 找到主文档并打开

- 在 VS Code 左侧文件栏，双击 `main.tex` 打开它。

你会看到类似这样的内容（模板默认给的）：

```
\documentclass[withoutpreface,bwprint]{commons/cumcmthesis}

% 导言区：用来加载宏包、自定义命令等
\input{commons/preamble.tex}

% 论文基本信息
\input{contents/info.tex}

\begin{document}
\maketitle

\input{contents/abstract.tex}

\input{contents/sections/s1.tex}
\input{contents/sections/s2.tex}

\input{contents/references.tex}

\newpage
\begin{appendices}
\input{contents/appendix/a1.tex}
\end{appendices}

\end{document}
```

1.2 编译它（一次就好，模板已配好）

► 方法 A：在命令行编译（推荐先学这个，最直观）

在 VS Code 里按 `Ctrl+``（反引号，键盘左上角 `Esc` 下面那个键）打开内置终端，然后输入：

```
latexmk main
```

看到最后出现类似 "Output written on main.pdf" 就很稳。也可以自己来一版更原始的命令（第一次这样）：

```
xelatex main
```

注意：如果只跑一遍 `xelatex`，交叉引用（比如「见图 1」）可能还是问号。所以推荐用 `latexmk`，它会自动帮你多跑几遍。

► 方法 B：用 **LaTeX Workshop** 的编译按钮（日常打码最方便）

- 打开 `main.tex`，点右上角的 ►（或按 `Ctrl+Alt+B`），选择 "`latexmk (xelatex)`"。
- 编译完成，VS Code 会弹出一个 PDF 预览标签页。

► 看结果

- 左侧点击 `main.pdf`（或点预览标签页），你就能看到：题目页 + 摘要 + 正文章节 + 参考文献 + 附录。

如果弹出红色报错面板，别慌——跳到 1.4 的「读报错心法」。

1.3 逐行读懂 `main.tex`（这是全课最重要的一段）

现在，我们把 `main.tex` 的骨架一行一行看懂，你就理解 LaTeX 是怎么工作了。

```
\documentclass[withoutpreface,bwprint]{commons/cumcmthesis}
```

这一行是「文档类声明」，告诉 LaTeX 用哪个模板、开哪些开关。`{}` 内是文档类文件位置（`commons/cumcmthesis`，`.cls` 后缀省略）；`[]` 内是选项：

选项	作用	什么时候用
<code>withoutpreface</code>	不生成承诺书与编号页	电子版提交（默认比赛要求）
<code>bwprint</code>	黑白打印	打印稿
<code>colorprint</code>	彩色打印	默认，可省略
<code>draft</code>	编译时不嵌入图片和代码，提速	写作过程中

小心：最终提交一定要去掉 `draft`，否则图片和代码不会被塞进 PDF，会严重影响成绩。

```
% 导言区：用来加载宏包、自定义命令等
\input{commons/preamble.tex}
% 论文基本信息
\input{contents/info.tex}
```

% 开头的是「注释」，LaTeX 会忽略它，只用来查看备注。`\input{文件名}` 是「把另一个 .tex 文件的内容，原样搬到这里」。所以这两行就是把导言区文件和信息文件合并进来。

```
\begin{document}
\maketitle
...
\end{document}
```

`\begin{document}` 到 `\end{document}` 之间，才是真正要输出到 PDF 的内容。`\maketitle` 命令在这里的作用是「输出封面/题目页」。标题和队员信息在哪定义？在 `contents/info.tex` 里（下一章就填它）。

```
\input{contents/abstract.tex}
```

引入摘要。想改摘要内容、加关键词，去 `contents/abstract.tex`。

```
\input{contents/sections/s1.tex}
\input{contents/sections/s2.tex}
```

引入正文章节。`s1.tex`、`s2.tex` 就是第一章、第二章……你可以按需增减行，或新建 `s3.tex`、`s4.tex` 并在 `main.tex` 里同样加一行 `\input` 把它接进来（后面 4.6 实战会演示）。

```
\input{contents/references.tex}
```

引入参考文献列表（第八章详解）。

```
\newpage
\begin{appendices}
\input{contents/appendix/a1.tex}
\end{appendices}
```

这是附录区。`\begin{appendices} ... \end{appendices}` 里放附录；`a1.tex` 是第一个附录文件（第九章详解）。

1.4 第一次报错怎么办（读报错心法）

报错不可怕，LaTeX 的报错信息其实很直白。你只要会看三样东西：

- 报错里第一句 "!" 开头的英文，往往就是问题描述。
- 冒号后面会给出出错的文件名和行号，比如 `main.tex:12`，就是第 12 行出问题。
- 最常见的几种：宏包没装（File `xx.sty` not found）、`\begin{...}` 少了对应的 `\end{...}`、写错了命令名（Undefined control sequence）。

► **小技巧：**在 **LaTeX Workshop** 里点报错，**VS Code** 会直接定位到出错行。改完再编译一次即可。

还没学到的地方先别乱改——后面每一章都会教你改对。这一章的目标只是「跑通 + 看懂骨架」。

第二章 填写论文基本信息 (info.tex)

目标：打开 `contents/info.tex`，把你的题目和队伍信息填进去，再编译看封面/题目页。这是你第一次真正改内容。

2.1 认识信息命令

打开 `contents/info.tex`，里面就是一组「赋值命令」。每个命令都是「`\命令名{值}`」的形式：

```
% 题目
\title{中国大学生数学建模竞赛 \LaTeX{} 模板}

% 如下信息在需要包含承诺书和编号专用页时用到
\tihao{A}           % 题号：从 A/B/C/D/E 选一个
\baominghao{1234}   % 报名号（12 位数字）
\schoolname{北京大学} % 学校全称
\membera{张三}      % 队员 1
\memberb{李四}      % 队员 2
\memberc{王五}      % 队员 3
\supervisor{赵六}   % 指导教师/负责人
\yearinput{2026}     % 年
\monthinput{8}       % 月
\dayinput{3}         % 日
```

2.2 实战：填成一支真实的队

► 把 `\title{...}` 里的内容改成你的论文题目。

例如你要写「基于遗传算法的物流路径优化」，就把那行改成：

```
\title{基于遗传算法的城市物流配送路径优化}
```

- ▶ 按实际情况填写 `\tihao`、`\schoolname`、`\membera`、`\memberb`、`\memberc`、`\supervisor`、`\yearinput`、`\monthinput`、`\dayinput`。

注意：`\title` 里的 `\LaTeX` 是「`\LaTeX` 这个 Logo」的专用命令。如果你想在标题里写「`LaTeX` 字样」的普通文字，直接敲 `LaTeX` 即可，不要加反斜杠。

2.3 重新编译看效果

- ▶ 回到 `main.tex`，按 **Ctrl+Alt+B** 或命令行 `latexmk main` 重新编译。
- ▶ 看 PDF 第一页：现在标题、题目、队员信息已经换成你的了。

`\tihao`、`\baominghao`、`\schoolname`、`\membera/b/c`、`\supervisor` 这些信息，只在「包含承诺书和编号页」时才用得到（即不带 `withoutpreface` 选项）。电子版通常不需要，但先填上无妨。

2.4 承诺书与编号页的开关（理解 `withoutpreface`）

想看看承诺书长什么样？把 `main.tex` 第一行的 `withoutpreface` 删掉再编译：

```
\documentclass[bwprint]{commons/cumcmthesis} % 会包含承诺书+编号页（纸质打印用）
```

想回到电子版：

```
\documentclass[withoutpreface,bwprint]{commons/cumcmthesis} % 电子版，不含封面承诺书
```

比赛电子版用 `withoutpreface`；纸质打印才把承诺书也印出。其它情况保持默认即可，不用纠结。

第三章 写摘要 (`abstract.tex`)

目标：学会用摘要环境和关键词命令，写一段「问题—方法—结果」式的建模摘要。摘要决定了评审的第一印象，值得单独一章。

3.1 摘要环境与关键词

打开 `contents/abstract.tex`，模板默认给你的：

```
\begin{abstract}
\textit{cumcmthesis} 是为\textbf{中国大学生数学建模竞赛}
```

(`\textbf{CUMCM}`) 编写的 `\LaTeX` 模板...

```
% 关键词
\keywords{\TeX{}\quad \LaTeX{}\quad 图片\quad 表格\quad 公式}
\end{abstract}
```

`\begin{abstract} ... \end{abstract}` 是「摘要环境」，里面写摘要正文。`\keywords{...}` 是关键词命令，花括号里用 `\quad`（一个较大空白）或逗号分隔多个关键词。

3.2 实战：写一段建模摘要

► 清空 `abstract.tex` 里的示例正文，改成你自己的摘要。

摘要公式：一段话 = ①这个问题解决什么 → ②用了什么模型/方法 → ③得到什么结果 → ④结论。例如：

```
\begin{abstract}
本文针对城市物流配送的路径优化问题，建立了基于遗传算法的车辆路径模型。首先，将客户点、配送点与道路转化为带权图，以配送总里程最短为目标，构造混合整数规划模型；其次，设计二进制编码与锦标赛选择策略，并引入局部搜索算子，提出了改进遗传算法进行求解；最后，利用某市 50 个配送点的实际数据，将本文算法与模拟退火算法对比，配送里程平均降低 7.3%，验证了模型的合理性与算法的高效性。

% 关键词
\keywords{车辆路径问题\quad 遗传算法\quad 车辆路径\quad 物流配送}
\end{abstract}
```

摘要里的 `\usepackage-style` 命令别加；摘要区域只放正文文字和 `\keywords`。想强调某个词用 `\textbf{...}` 加粗即可（下一章会讲）。

3.3 摘要的常见毛病（评审视角）

- 不要出现「本文先...再...然后...」这种流水账，直接说模型和结果。
- 结果一定要给「数字 + 结论」，比如「里程降低 7.3%」，比「效果更好」有说服力。
- 摘要里出现的符号，别指望读者看正文才知道含义；最好在摘要里就直接给出定义或用中文说明。

第四章 正文：分节与文字排版

目标：把论文的「骨架」搭起来——学会分章节、写段落、切字体、列要点。做完这章，正文的框架就成型了。

4.1 章节命令与自动编号

LaTeX 的章节命令分三级，自动编号、自动排进目录：

命令	层级	编号样子
<code>\section{...}</code>	一级（章）	一、二、三...（中文序号）
<code>\subsection{...}</code>	二级（节）	1.1、1.2...
<code>\subsubsection{...}</code>	三级（小节）	1.1.1...

新建一个正文文件 `contents/sections/s3.tex`，写：

```
\section{问题分析}
\subsection{背景与现状}
...正文文字...

\subsection{问题提炼}
...正文文字...
```

▶ 然后在 `main.tex` 的正文区加一行 `\input{contents/sections/s3.tex}`，放在 `s2.tex` 之后。再编译，你就会看到「一、问题分析」这样的一级标题，以及自动编号的二级标题。

4.2 段落、缩进、换行

一段就是一「段」：LaTeX 里，一个自然段 = 一段连续文字，段与段之间用「一个空行」隔开。首行缩进模板已自动处理（两个汉字）。

这是第一段的第一句话。继续写下去.....这一行的末尾不要敲回车，直接接着写。

（空一行之后才是第二段）第二段会自动另起一行，并自动首行缩进。

- 强制换行（另起一行但不另起一段）：`\\`（两个反斜杠）
- 想要明显的段间距，可以在段落前后用 `\bigskip` 或 `\medskip`。
- 个别地方想另起一行并保持同一段，用 `\\`。

4.3 字体与强调

`cumcm` 模板已经帮你规定好了正文用宋体，并提供几个现成的字体命令：

命令	效果	用途
<code>\textbf{文字}</code>	加粗（黑体）	强调

<code>\heiti</code> 文字	黑体	标题/强调
<code>\song</code> 文字	宋体	正文
<code>\kai</code> 文字	楷体	引用/提示
<code>\textit{文字}</code>	斜体（英文）	英文强调

```
\textbf{这是加粗的文字。}  
\kai 这是楷体提示文字。  
\textit{This is italic.}
```

4.4 列表

无序列表 `itemize`、有序列表 `enumerate`：

```
无序列表：  
\begin{itemize}  
  \item 第一点  
  \item 第二点  
\end{itemize}  
  
有序列表：  
\begin{enumerate}  
  \item 第一步  
  \item 第二步  
\end{enumerate}
```

4.5 注释与脚注

`%` 是注释（编译时忽略，用来给自己做备注）。脚注用 `\footnote{...}`：

```
% 这是一行注释  
正文内容\footnote{这是脚注，补充说明一些东西。}。
```

4.6 实战：搭好整篇论文的框架

► 在 `contents/sections/` 里按常规建模结构建立章节文件（`s3.tex`、`s4.tex...`），每章做一件事。

一个典型建模论文的章节骨架：

```
\section{问题重述}  
\section{问题分析}
```

```
\section{模型假设}
\section{符号说明}
\section{模型的建立与求解}
\section{模型的检验与灵敏度分析}
\section{模型的评价与改进}
```

► 在 `main.tex` 正文区，把每个章节文件都 `\input` 进来。编译后，目录和章节标题就都有了。

框架比内容先定型，是写建模论文的好习惯——先把「骨架」立住，再逐章填「肉」。

第五章 插图 (figures/)

目标：把你的图放进论文，并学会让 LaTeX 自动编号、自动生成交叉引用。建模论文里图很多，这是高频操作。

5.1 图片放哪、用什么格式、怎么命名

- 所有图放进 `figures/` 文件夹。模板已经设置了图片搜索路径就是 `figures/`，所以正文里 `\includegraphics{名字}` 会去那里找。
 - 位图推荐 `jpg`、`png`；矢量图推荐 `pdf`。避免 `bmp` / `eps`，容易出问题。
 - 命名用英文+数字，要「有意义」，别用 `1.png`、`2.png`。例如 `route_optimization.pdf`、`map_50_points.png`。
- 模板支持的图片后缀顺序：`.pdf`、`.eps`、`.jpg`、`.png`。

5.2 插入一张图的基本写法

新建一个图片环境 `figure`，包含 `\includegraphics` + `\caption` + `\label`：

```
\begin{figure}[!htbp]
\centering
\includegraphics[width=0.6\textwidth]{route_optimization.pdf}
\caption{优化后的配送路径示意}
\label{fig:route}
\end{figure}
```

代码	解释
<code>\begin{figure}[!htbp]</code>	开始一个图环境。 <code>[]</code> 里告诉 LaTeX 图放哪：h=here 此处、t=top 页顶、b=bottom 页底、p=单独占一页；! 表

	示尽量忽略默认限制
<code>\centering</code>	让图在页面上居中
<code>\includegraphics[width=0.6\textwidth]{route_optimization.pdf}</code>	插入图片。 <code>width=0.6\textwidth</code> 表示图片宽度=正文宽度的 60%
<code>\caption{优化后的配送路径示意}</code>	图的标题（自动编号为「图 1」）
<code>\label{fig:route}</code>	给这张图起个引用名（标签）

5.3 在正文里引用这张图（交叉引用）

模板用 `cleveref` 宏包，交叉引用用 `\cref{标签}`，会自动生成「图 1」这样的中文引用：

优化后的配送路径如图~`\cref{fig:route}` 所示。

保险起见，引用标签必须是唯一的。你之前 `\label{fig:route}`，这里就 `\cref{fig:route}`；标签写什么由你自己定，但正文和 `\label` 要完全一致。

5.4 多张图并排（`minipage` + `subcaption`）

想做「两张小图并排放一起，下面一个大标题」：用 `minipage` 环境，每个小图用 `\subcaption` 编号。

```
\begin{figure}[!htbp]
  \centering
  \begin{minipage}[c]{0.45\textwidth}
    \centering
    \includegraphics[width=\textwidth]{before.png}
    \subcaption{优化前}
    \label{fig:before}
  \end{minipage}\hfill
  \begin{minipage}[c]{0.45\textwidth}
    \centering
    \includegraphics[width=\textwidth]{after.png}
    \subcaption{优化后}
    \label{fig:after}
  \end{minipage}
  \caption{优化前后配送路径对比}
  \label{fig:compare}
\end{figure}
```

引用大图用 `\cref{fig:compare}`，引用小图用 `\cref{fig:before}`、`\cref{fig:after}`。如果两张图高度不一样、想等高，把 `\includegraphics[width=..]` 改成

`\includegraphics[height=0.2\textheight]` 即可。多张图并排时一定要用两个 `minipage` 各装一张，中间用 `\hfill` 把二者隔开。

模板已加载 `subcaption` 宏包，`\subcaption` 可直接用，无需再加载。

5.5 浮动体的脾气

`figure` 是「浮动体」：LaTeX 有可能为了让版面更美观，把图挪到本页的其它位置，甚至下一页。所以别指望「图一定出现在你写的上一行正下方」。标签和引用是自动对齐的，挪了位置编号也不变，这就是用 LaTeX 图的好处。

第六章 公式：建模的主战场

目标：学会建模论文里最常用的公式写法——行内、行间、编号、对齐、矩阵、分段函数、粗体符号，以及定理/证明环境。

6.1 行内公式

用 `$ ... $` 把公式嵌在文字中间：

设 x 为配送量， c_{ij} 表示两点间每单位运输成本。

6.2 行间公式与编号

不带编号、独占一行的公式，用 `\[ ... \]`：

```
\[
E = mc^2
\]
```

带编号、方便引用的，用 `equation` 环境：

```
\begin{equation}
z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}
\label{eq:objective}
\end{equation}
```

引用它：

目标函数如式~\cref{eq:objective} 所示。

注意：正文里引用公式用 `\cref`，自动显示为「式 (1)」这类，比手动写编号省心得多，也绝不会错。

6.3 多行公式对齐 (align)

多行公式想在某处对齐，用 `align` 环境，`&` 标出对齐位置：

```
\begin{align}
P &= UI \\
&= I^2R
\end{align}
```

6.4 矩阵

矩阵用 `pmatrix`（圆括号）、`bmatrix`（方括号）、`vmatrix`（竖杠）等环境（来自 `amsmath`，模板已加载）：

```
\[
\mathbf{A} = \begin{pmatrix}
a_{11} & a_{12} & a_{13} \\
a_{21} & a_{22} & a_{23} \\
a_{31} & a_{32} & a_{33}
\end{pmatrix}
\]
```

6.5 分段函数 (cases)

`cases` 环境要放在数学环境里：

```
\[
f(x) = \begin{cases}
0, & x \text{ 为无理数}, \\
1, & x \text{ 为有理数}.
\end{cases}
\]
```

6.6 常用数学细节

- 希腊字母：`\alpha`、`\beta`、`\gamma`、`\theta`、`\lambda`、`\mu`、`\sigma`、`\Delta`...
- 上标下标：`x^2`、`x_{ij}`；分数：`\frac{a}{b}`；求和：`\sum_{i=1}^n`；积分：`\int a^b`。
- 公式里要写中文/文字，用 `\text{...}`（否则字体是数学斜体）：

```
x \text{ 为配送点数量}
```

- 需要加粗的数学符号（向量、矩阵），用 `\bm{...}`（来自 `bm` 宏包）：

`\bm{\alpha}` 是粗体希腊字母, `\bm{x}` 是粗体向量。

6.7 定理 / 定义 / 证明环境

模板内置了一整套数学环境, 自动编号、自动生成中文标题: `definition` (定义)、`theorem` (定理)、`lemma` (引理)、`corollary` (推论)、`assumption` (假设)、`conjecture` (猜想)、`axiom` (公理)、`principle` (定律)、`problem` (问题)、`example` (例)、`proof` (证明)、`solution` (解)。

```
\begin{assumption}
假设各配送点的需求量在计划期内保持不变。
\label{ass:demand}
\end{assumption}
```

```
\begin{theorem}
模型的最优解存在且可在有限步内求得。
\label{thm:exist}
\end{theorem}
```

```
\begin{proof}
(这里写证明过程。)
\end{proof}
```

引用它们同样用 `\cref{标签}`:

由假设~`\cref{ass:demand}` 与定理~`\cref{thm:exist}` 可知.....

6.8 实战: 写一个目标函数 + 约束

把前面学的拼起来, 写一个典型的优化模型:

```
\begin{assumption}
配送中心与各需求点均视为平面上的点。
\label{ass:point}
\end{assumption}

目标函数:

\begin{equation}
\min\; Z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}
\label{eq:obj2}
\end{equation}
```

约束条件:

```
\begin{align}
& \sum_{j=1}^n x_{ij} \leq Q_i, \quad i=1,\ldots,n \quad \text{\label{eq:cap}} \\
& x_{ij} \geq 0 \quad \text{\label{eq:int}} \quad \text{\text{且为整数}}
\end{align}
```

其中, c_{ij} 为两点间单位运输成本, x_{ij} 为配送量, Q_i 为第 i 个点的容量上限。目标函数~\cref{eq:obj2} 表示配送总成本最小, 约束~\cref{eq:cap} 为容量约束, 约束~\cref{eq:int} 为整数约束。

第七章 三线表: CUMCM 的标准表格

目标: 学会画比赛要求的三线表。注意: CUMCM 明确要求表格是「三线表」, 不要画带竖线的表格(那种常被扣分)。

7.1 什么是三线表

三线表只有三条横线: 顶线(\toprule)、表头下的分线(\midrule)、底线(\bottomrule)。没有竖线。它由 booktabs 宏包提供, 模板已加载。

7.2 一个完整的三线表

```
\begin{table}[!htbp]
\centering
\caption{三种算法求解结果对比}
\label{tab:compare}
\begin{tabular}{ccc}
\toprule[1.5pt]
算法 & 总里程(km) & 平均耗时(s) \\
\midrule[1pt]
标准遗传算法 & 135.4 & 42.1 \\
改进遗传算法 & 125.2 & 38.6 \\
模拟退火算法 & 143.8 & 51.7 \\
\bottomrule[1.5pt]
\end{tabular}
\end{table}
```

引用它:

由表~\cref{tab:compare} 可见，改进遗传算法总里程最短。

7.3 看懂 tabular 的列格式

列格式字符	含义
c	列内容居中
l	列内容左对齐
r	列内容右对齐
	画竖线（建模论文表格不要用！）

```
\begin{tabular}{ccc}          % 三列，都居中
表头 1 & 表头 2 & 表头 3 \\          % 每行用 \\ 结束，列之间用 & 分隔
数据 1 & 数据 2 & 数据 3 \\
\end{tabular}
```

`\toprule[1.5pt]` 画顶线（线宽 1.5pt）；`\midrule[1pt]` 画表头下分线（线宽 1pt）；`\bottomrule[1.5pt]` 画底线。表格的标题用`\caption`，模板规定表格标题放在表格上方（和图相反）。

7.4 实战与禁忌

- ▶ 做一张结果对比表（如上面 7.2 的示例），把数据换成你模型算出来的。
- ▶ 把表格放到对应章节，用`\cref`在正文里引用。

禁忌：不要用`\hline`和`|`画竖线表格；不要用「去掉表格标题」；表格标题写清「表：内容」即可。数据表最好加单位（如 km、s），并保持小数位一致，更专业。

第八章 参考文献 + Zotero 详细手把手

目标：学会在 cumcm 模板里引用文献，并把 Zotero 完整地用起来。这一章是「实战 + 工具」双料章节，内容较多，建议对照电脑一步一步做。

8.1 模型里参考文献是怎么写的

cumcm 模板用最经典、最稳的方式：`thebibliography` 环境 + `\bibitem` 逐条列出，不用外部的 `biblatex/biber`。打开 `contents/references.tex`，默认是这样的：

```
\begin{thebibliography}{9}
  \bibitem[1]{liuhaiyang2013latex} 刘海洋.
  \newblock \LaTeX{}入门\allowbreak[J].
  \newblock 电子工业出版社, 北京, 2013.
  \bibitem[2]{mathematical-modeling} 全国大学生数学建模竞赛论文格式规范 (2026 年修改).
\end{thebibliography}
```

代码	作用
<code>\begin{thebibliography}{9}</code>	开始参考文献环境；{9} 表示编号最多两位（够用即可）
<code>\bibitem[1]{liuhaiyang2013latex}</code>	定义一条文献；[1] 是显示编号，{liuhaiyang2013latex} 是引用键（key）
<code>\newblock</code>	分隔文献的各个字段（作者/题名/来源等）
<code>\end{thebibliography}</code>	结束

8.2 在正文里引用

模板提供了三种引用命令：

命令	效果	用在哪
<code>\cite{key}</code>	一般插入式编号（如 [1]）	普通描述
<code>\upcite{key}</code>	上标引用（推荐，比赛常用）	句末上标 [1]
<code>\supercite{key}</code>	更小更靠上的上标	极少用

正文举例：

```
遗传算法由 Holland 提出\upcite{holland1975}，后续被广泛用于路径优化问题
\upcite{liu2020}。
```

8.3 方法 A：手写参考文献（最简单、完全可控）

在 contents/references.tex 里，把每条参考文献写成一行 \bibitem。

编排格式参照 GB/T 7714：

```
\begin{thebibliography}{9}
  \bibitem{holland1975} Holland J H.
  \newblock Adaptation in Natural and Artificial Systems[M].
  \newblock MIT Press, 1992.
  \bibitem{liu2020} 刘宏, 王强.
```

```
\newblock 带时间窗的车辆路径问题改进算法[J].  
\newblock 系统工程理论与实践, 2020, 40(5): 1205-1215.  
\end{thebibliography}
```

- ▶ 在正文里用 `\upcite{key}` 引用, `key` 要和 `\bibitem` 后面的 `key` 完全一致。

优点: 零依赖、格式完全受控、编译最简单。缺点: 手动敲, 稍麻烦。适合文献量不大时用。

8.4 方法 B: 用 bibtex 自动生成 (文献多时推荐)

原理: 你只维护一个 `.bib` 文献库文件, 正文用 `\upcite{key}` 引用; `bibtex` 自动把被引用的条目按顺序编好号。

- ▶ 把 `contents/references.tex` 的内容替换为:

```
\bibliographystyle{unsrt} % 按被引用的先后顺序编号 (比赛最常用)  
\bibliography{refs} % 不带 .bib 后缀, 指向根目录的 refs.bib
```

- ▶ 把 `Zotero` 导出的 `refs.bib` 放到项目根目录 (和 `main.tex` 同级)。
- ▶ 编译顺序: `xelatex main` → `bibtex main` → `xelatex main` → `xelatex main`。

如果你用 `latexmk`, 它会自动检测并完成这些步骤, 直接:

```
latexmk main
```

想要中文标准 GB/T 7714 的样式, 可在 `preamble.tex` 里加载 `%\usepackage[numbers]{gbt7714}` (需额外安装), 再把 `\bibliographystyle` 设为对应样式。这里先不展开。

8.5 Zotero 详细从零配置 (手把手)

`Zotero` 是免费的文献管理软件, 目的: ①保存你找到的文献; ②自动生成引用编号; ③导出成 `refs.bib` 给 `cumcm` 用。下面从零开始。

8.5.1 下载与安装 Zotero 7

- ▶ 打开 zotero.org, 点 **Download**, 下载安装包 (Windows / macOS 都支持)。
- ▶ 按默认安装即可。装完打开, 界面是: 左侧是资料库 (各类收藏夹), 中间是文献列表, 右边是细节信息 (作者、标题、摘要等)。

8.5.2 添加文献 (两种最常用方式)

- ▶ 方式一: 浏览器插件一键抓取 (最常用)

- 给浏览器装 Zotero Connector 扩展（Chrome/Firefox/Edge 都支持，在 Zotero 官网下载）。
- 在论文页面（知网、Springer、Google Scholar 等），点浏览器右上角的 Zotero 插件图标，它会自动抓取题录信息存进 Zotero。
- 如果是 PDF 页面，插件可能还会帮你把附件 PDF 一起存进来。
- ▶ **方式二：手动添加 / 用标识号添加**
 - 点工具栏的「+」→ 新建条目 → 选类型（期刊文章、图书、学位论文等）→ 填字段。
 - 或点「由标识符添加」，粘贴 DOI / ISBN，Zotero 自动补齐信息。
- ▶ **给文献建立分类（Collection）**
 - 左侧「我的资料库」上右键 → 新建分类 → 取名（如「本论文参考文献」），把用到的文献拖进去。

8.5.3 安装 Better BibTeX 插件 (关键步骤)

Zotero 默认导出的 .bib 的引用键（key）不符合 LaTeX 习惯。Better BibTeX（简称 BBT）能让你用「作者_年份」这种规则自动生成规范的 key，并且支持「保持更新」自动同步到文件。

- ▶ **下载 Better BibTeX:** 到 GitHub (retorquere/zotero-better-bibtex) 的 Releases 页，下载最新的 .xpi 文件。
- ▶ **安装:** 在 Zotero 里「工具 → 插件」→ 右上角齿轮 → 「从文件安装插件...」→ 选中刚才下载的 .xpi → 安装 → 重启 Zotero。

Zotero 7 也可在「工具 → 添加到 Zotero 插件」里搜 better-bibtex 一键安装。

8.5.4 配置引用键 (Citation Key) 格式

- ▶ 打开 Zotero → 编辑 → 设置 → Better BibTeX → Citation Keys。
- ▶ 把「Citation key 格式」设为 [auth:lower]_[year]，例如 smith_2023（小写作者名_年份）。
- ▶ 勾选「固定引用键」（pin citation keys），这样 key 不会因为文献增删而变。

这样每条文献都有类似 smith_2023 的 key，直接可以用作 \bibitem 的 {key} 或 \upcite{key}。要查看某条的 key：右键条目 → 「复制引用键」，或者直接看绿色角落的引用键标签。

8.5.5 导出 .bib 文件 (支持自动更新)

- ▶ 在左侧选中你的分类（Collection）→ 右键 → 「导出资料库...」。
- ▶ 格式选「Better BibTeX」，并勾选「保持更新」（Keep updated）。
- ▶ 存成 refs.bib，放到 cumcm 项目根目录。

打开 refs.bib 看看，里面每条长这样（Zotero 自动生成的，你基本不用改）：

```
@article{liu2020,
  author = {刘宏 and 王强},
  title = {带时间窗的车辆路径问题改进算法},
  journal = {系统工程理论与实践},
  volume = {40},
  number = {5},
  pages = {1205--1215},
  year = {2020}
}
```

到这里，Zotero 就完成接入：以后在 Zotero 里加文献 → refs.bib 自动更新 → 在 cumcm 里 `\upcite{key}` 引用，全程不用手动改 bib。

8.5.6 在 cumcm 里成功用上 Zotero 文献（完整闭环）

- ▶ 确认 refs.bib 在项目根目录（和 main.tex 同级）。
- ▶ 把 references.tex 改成 8.4 的写法（`\bibliographystyle{unsrt} + \bibliography{refs}`）。
- ▶ 在正文某处用 `\upcite{liu2020}` 引用。
- ▶ 编译：`latexmk main`；或依次 `xelatex → bibtex → xelatex → xelatex`。
- ▶ 看 PDF 参照文献那一页，应该出现了编号 [1] 且正文有上标引用。

若正文有引用却没出现在文献表，或出现「？」，多半是：①编译没跑够遍数；②key 拼错；③refs.bib 路径不对。按 8.7 排查。

8.6 两种方法的取舍

方式	优点	缺点	适合
A 手写 thebibliography	零依赖、可控、编译最简单	手动维护，文献多时累	文献少（<10 条）或结构固定
B bibtex + refs.bib	Zotero 自动同步、改文献方便	编译顺序多一步	文献多、常用 Zotero

8.7 参考文献常见问题排查

- 问题「？」：正文引用了key，但参考文献里没有。检查key是否完全一致、是否真编译了 bibtex。
- 问题「未定义引用」：编译遍数不够，多跑一遍 latexmk 即可。
- 问题「\addbibresource 未定义」：说明还在用 biblatex 的命令。cumcm 不带 biblatex，请改用 \bibliography 或手写 thebibliography。
- 文献表编号不是按引用顺序：改用 \bibliographystyle{unsrt}（unsorted）。

第九章 附录：放代码和算法

目标：把源程序、算法伪代码放到附录里，并让代码带语法高亮。

9.1 附录环境

附录区在 main.tex 里已经用 \begin{appendices} 包起来了。你只需要往 contents/appendix/ 里加文件并在 main.tex 里 \input 它，或直接打开已有的 a1.tex 等。

```
\begin{appendices}
\input{contents/appendix/a1.tex}
\input{contents/appendix/a2.tex}
\end{appendices}
```

9.2 用 Istlisting 放代码（带高亮）

模板已加载 listings 宏包，并用 \lstset 配好了常用的代码高亮。直接：

```
\section{求解算法 MATLAB 代码}

\begin{lstlisting}[language=matlab]
% 蒙特卡洛估算圆周率
N = 1e6;           % 随机点数量
x = rand(N,1);
y = rand(N,1);
inside = (x.^2 + y.^2) <= 1;
pi_est = 4 * sum(inside) / N;
fprintf('估算的值: %.6f\n', pi_est);
\end{lstlisting}
```

把 language=matlab 换成 language=C、language=python、language=TeX 等可切换语言。

如果你要放 C / Python 等非 MATLAB，模板默认的 `\lstset` 不一定预配置了对应关键字颜色，但基本高亮仍可用。实在要高级配色，可在 `preamble.tex` 里再用 `\lstset` 细化。

9.3 实战

- ▶ 把一个附录文件（如 `a2.tex`）改成你自己的算法/源程序，用 `lstlisting` 包起来。
- ▶ 在 `main.tex` 里确认 `\input` 了它，重新编译，看附录里的代码是否排版整洁。

附录里的代码要能读懂，行数多的拆成几段放；不要图省事把整个代码全塞一段，评审会看花眼。

第十章 打磨与提交（收尾）

目标：把论文从「写出来了」打磨到「能交」。重点是去掉临时选项、检查图片代码是否嵌入、确认格式合规。

10.1 决定要哪些选项

回顾 `main.tex` 第一行。最终提交时：

- 去掉 `draft`：让图片和代码真正嵌入 PDF。
- 电子版：保留 `withoutpreface`（不含承诺书和编号页）。
- 黑白/彩色：按比赛要求用 `bwprint` 或 `colorprint`（默认彩色）。

10.2 检查清单（提交前逐项打勾）

- 编译用 `xelatex` 跑通，无红色报错。
- 去掉 `draft` 后再编译一次，确认 PDF 里图片和代码都出现了。
- 目录、图表、公式、参考文献编号和交叉引用都是自动编号，无「？」或「??」。
- 所有图都在 `figures/`，文件名英文、无空格、有意义。
- 所有表格都是三线表（`booktabs`），没有竖线表格。
- 摘要和关键词齐全，摘要独立可读、有数字结论。
- 参考文献用 `\upcite` / `\bibliography` 正常引用，格式（GB/T 7714 风格）统一。
- 附录里放了关键源程序/算法，代码高亮正常。
- 用 `latexmk -C main` 清理临时文件，重新生成干净的 PDF 再提交。

10.3 常见报错速查（FAQ）

报错/现象	原因	解决
File 'xxx.sty' not found	宏包没装或拼错	tlmgr install xxx; 注意模板已加载的宏包不要重复加载
必须使用 xelatex	用错了编译器	改用 xelatex 或 latexmk
中文乱码/空白	编译器不对或字体缺失	用 xelatex; 确保 fonts/ 存在; 文件另存为 UTF-8
参考文献不显示/?	没跑 bibtex 或 key 不一致	编译 4 遍或用 latexmk; 核对 \bibitem 的 key
图/表格没出现	位置浮动或用了 draft	去掉 draft; 必要时改用 [!htbp]
\begin/end 不匹配	环境配对错	数一下 \begin{...} 与 \end{...} 是否对应
Undefined control sequence	命令名写错	检查命令拼写; 该宏包没加载

10.4 用 AI 辅助提速 (可选)

VS Code 里的 Codex / Copilot 等插件, 可以在 .tex 里写注释描述需求, 让 AI 生成 LaTeX 代码。但它们经常用错宏包或命令, 务必把 AI 当「初稿生成器」, 再按本手册的规范微调。

- 让 AI 生成一个三线表、一段公式、一个 lstlisting 代码块, 都是它的强项。
- AI 默认可能用竖线表格、用 biblatex 命令——这些在本模板里都不合规/不可用, 注意纠正。

10.5 学习资源

- cumcm 模板 README: <https://github.com/jayxin/cumcm>
- 全国大学生数学建模竞赛官网: <http://www.mcm.edu.cn>
- 论文格式规范: 模板 docs/ 目录下的《全国大学生数学建模竞赛论文格式规范-2026 年修订稿.pdf》
- Overleaf 文档: <https://www.overleaf.com/learn>
- LaTeX Wikibook: <https://en.wikibooks.org/wiki/LaTeX>
- CTAN: <https://www.ctan.org/> (查宏包文档)
- Detexify: <http://detexify.kirelabs.org/> (手写识别 LaTeX 符号)

结语

现在你走完了从「不会 LaTeX」到「能完整写出一篇建模论文」的全流程：

- ☒ 理解了 LaTeX「源码 → 编译 → PDF」的工作方式
- ☒ 熟悉了 cumcm 模板的文件结构与主要选项
- ☒ 会写摘要、正文、插图、公式、三线表、附录
- ☒ 会用 `\cref` `\upcite` 自动编号与交叉引用
- ☒ 会用 Zotero + Better BibTeX 管理并接入参考文献

接下来你的任务只有一个：打开 `main.tex`，运行 `latexmk main`，开始写属于你的 CUMCM 论文！